

**LINUX
SCENARIO
BASED Q&A
DAY 10**

Q1. Schedule Heavy Cronjob During Off-Peak Hours

🧩 Scenario:

A resource-intensive task like a backup job must run only during off-peak hours to avoid affecting system performance.

🧠 Possible Root Cause:

The cron job is running during high-load periods, consuming CPU, I/O, or network bandwidth and affecting performance for active users.

🔧 Solution:

1. Schedule the job for early morning (e.g., 2 AM):

```
0 2 * * * /path/to/backup.sh
```

✅ Tip:

Always schedule backups, updates, and maintenance scripts during the system's least active period to reduce performance impact and avoid user disruption.

Q2. Temporarily Disable a Cronjob Without Deleting It

🧩 Scenario:

You want to pause a scheduled cronjob without permanently removing it from the crontab.

🧠 Possible Root Cause:

- The script is causing high resource usage or unexpected errors.
- The cronjob is no longer needed temporarily due to a change in business needs or maintenance.

🔧 Solution:

- Simply comment out the cron entry by adding a # at the beginning of the line.

Example:

```
# 0 3 * * * /path/to/script.sh # Temporarily disabled, under maintenance
```

✅ Tip:

Always include a brief comment explaining why the job was disabled to avoid confusion later.

Q3. Cronjob Scheduled But Disappears After Reboot

🧩 Scenario:

You've added cronjobs, but after a server reboot, they are missing or no longer run.

🧠 Possible Root Cause:

- Crontab was edited improperly or not saved correctly.
- Server uses an ephemeral or non-persistent filesystem (common in cloud VMs).
- Cron file was manually edited in /var/spool/cron/ instead of using crontab.

🔧 Solution:

1. Always use crontab -e to edit and save cronjobs.
2. Avoid manually modifying files under /var/spool/cron/username.
3. Ensure persistence: Verify the cron directory isn't on a temporary or non-persistent disk.

✅ Tip:

In cloud environments, confirm that the cron configuration persists across reboots by testing a reboot and checking crontab -l afterward.

Q4. Create a Compressed .tar.gz Backup

🧩 Scenario:

You need to generate a compressed archive of a directory or set of files for backup purposes.

🧠 Possible Root Cause:

Backups are failing or missing due to:

- Lack of compression causing large file sizes
- Manual copy methods not preserving directory structure or metadata
- Inconsistent naming making it hard to manage or rotate backups

🔧 Solution:

1. Use the tar command with compression options:

```
tar -czvf backup.tar.gz /path/to/data
```

Flag Breakdown:

- c → Create a new archive
- z → Compress using gzip
- v → Verbose output (shows progress)
- f → Output to specified file

✅ Tip:

Include the current date in the filename for easy version tracking:

```
tar -czvf backup-$(date +%F).tar.gz /path/to/data
```

Q5. Restore a Single File from a .tar.gz Archive

🧩 Scenario:

You need to extract just one specific file from a large compressed archive without unpacking everything.

🧠 Possible Root Cause:

- Unaware of the specific file path inside the archive.
- Attempting full extraction when only one file is needed, which wastes time and space.
- Risk of overwriting files due to lack of preview.

🔧 Solution:

1. Preview the archive contents:

```
tar -tzf backup.tar.gz
```

2. Extract the specific file:

```
tar -xzf backup.tar.gz path/to/neededfile
```

Replace path/to/neededfile with the exact path as shown in the preview list.

✅ Tip:

Always use the -t (list) option first to identify the file path and avoid accidental overwrites.

Q6. Unable to Extract, Tar Backup Appears Corrupted

🌿 Scenario:

Attempting to extract a .tar.gz archive fails partway with an error like:
Unexpected EOF or gzip: unexpected end of file.

🧠 Possible Root Cause:

- The backup was only partially created or transferred.
- Disk space ran out during compression.
- Network interruption occurred during file copy or download.

🔧 Solution:

1. Check the file size and integrity:

Ensure the .tar.gz file is fully present and not truncated.

```
ls -lh backup.tar.gz
```

2. Recreate the backup properly:

```
tar -czvf newbackup.tar.gz /data
```

✅ Tip:

Always verify backups right after creation using:

```
tar -tzf backup.tar.gz > /dev/null && echo "Backup OK"
```

Q7. Scheduling Daily Automated Backups with rsync

🌿 Scenario:

You need to automate daily backups of important data using rsync.

🧠 Possible Root Cause(when automation fails):

- cron service not running or misconfigured
- Incorrect rsync path or missing executable
- Permissions issue with source/destination directories
- Network mount not available at the time of backup (e.g., NFS/SMB delays)

🔧 Solution:

1. Set up a cron job to run daily at 2 AM:

```
0 2 * * * rsync -avz /data/ /backup/ >> /var/log/rsync-backup.log 2>&1
```

- -a: Archive mode (preserves permissions, timestamps, etc.)
- -v: Verbose output
- -z: Compress during transfer
- >> /var/log/rsync-backup.log 2>&1: Redirects both output and errors to a log file

✅ Tip:

Always log backup operations for later review. Use cron logs (/var/log/cron or journalctl) if needed to debug failures.

Q8. Hidden Files Missing After rsync

🧩 Scenario:

After running an rsync backup, hidden files (dotfiles like .bashrc, .env) are not copied to the destination.

🧠 Possible Root Cause:

- Missing trailing slash on the source path
- Improper include/exclude patterns
- Dotfiles may be skipped if not explicitly matched by a pattern

🔧 Solution:

- Always include a trailing slash in the source path to copy contents correctly:
`rsync -avz /src/ /dest/`

`/src/` copies the contents of `/src`, including hidden files. `/src` (without slash) behaves differently.

✅ Tip:

Use `-a` (archive) to preserve dotfiles and attributes. When in doubt, run a dry-run with `-n`:

```
rsync -avzn /src/ /dest/
```

This will preview the transfer without making changes.

Q9. Tar Extraction Overwrites Existing Files

🧩 Scenario:

Extracting a .tar.gz backup overwrites current files in the working directory.

🧠 Possible Root Cause:

- Tar extracts files into the current directory by default, replacing files with matching names.
- No path isolation used during extraction.

🔧 Solution:

- To avoid overwriting existing files, extract into a new directory:
`mkdir /tmp/restore`
`tar -xvf backup.tar.gz -C /tmp/restore/`

✅ Tip:

In production or critical environments, always extract backups to a separate directory first and review contents before merging.

Q10. Encrypting a Tar Backup for Secure Storage

🧩 Scenario:

You need to back up important data and ensure it is encrypted for secure storage or transfer.

🧠 Possible Root Cause:

- Storing sensitive backups without encryption can lead to data breaches if unauthorized access occurs.
- Backups transferred over insecure channels are vulnerable to interception.

 Solution:

1. Create a compressed backup:

```
tar -czf backup.tar.gz /data
```

2. Encrypt the archive using GPG (symmetric encryption):

```
gpg -c backup.tar.gz
```

(You'll be prompted to enter a passphrase for encryption.)

 Tip:

- To automate secure backups (e.g., in cron jobs), use:

```
gpg --batch --yes --passphrase-file /secure/path/pass.txt -c backup.tar.gz
```
- Ensure the passphrase file is readable only by root or the backup user:

```
chmod 600 /secure/path/pass.txt
```